RICE UNIVERSITY

Capstone: controlled experiments

In pairs, answer two experimental questions about the pipeline with fair, repeated measurements.

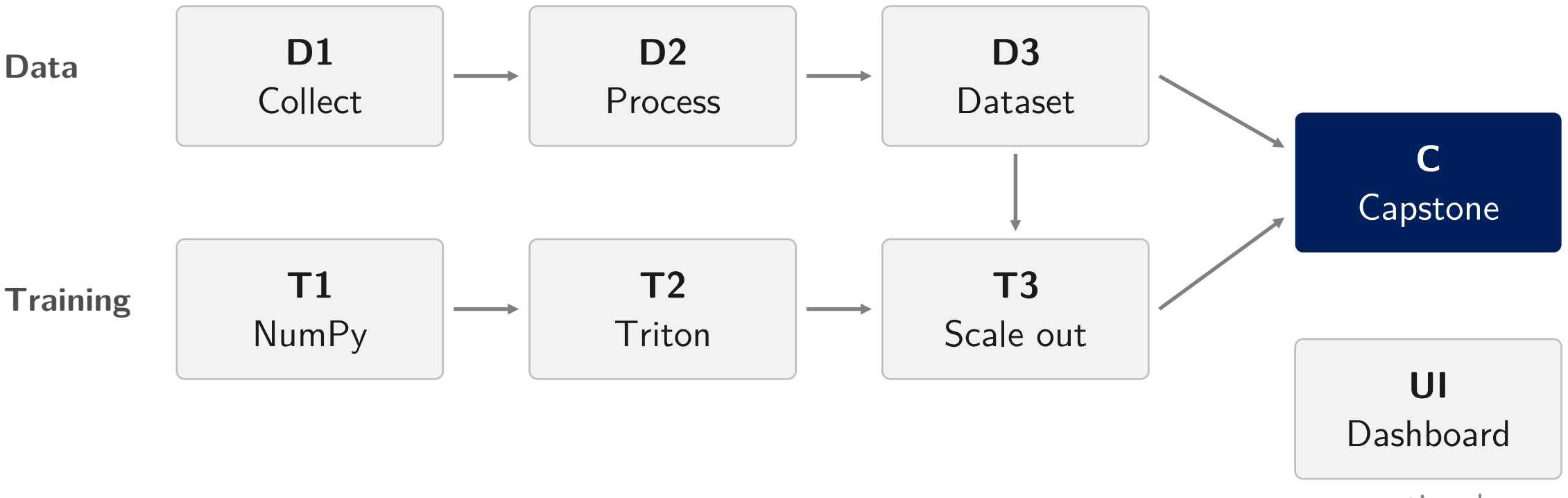
COMP 329 course staff

Rice University

Spring 2027 · Capstone

Released week 7 · Pairs set week 13 · Due May 4, 2027 · 13% of grade

Where this assignment fits



In our pipeline

This assignment: `D1-D3, T1-T3` → **C** → `6 pages + raw repeats`

What you will build, and why

Ask two clear questions, keep everything else fixed, and let repeated measurements answer them.

1. **No new algorithm**

   The starter already runs; the work is the experiment design.

2. **Controls**

   A fair comparison changes one thing at a time.

3. **Honesty**

   No speedup or no improvement can be a strong result.

What goes in, what comes out

Your six components
`D1-D3, T1-T3`
Staff outputs allowed after deadlines

→

C
Capstone

→

Pair report
`6 pages + raw repeats`
Plus a 5-minute demo

Files you produce

`One-page plan`
Questions, predictions, metrics, controls

`Raw repeats`
Configs, checksums, one row per measurement

`Report + demo`
Two comparisons with uncertainty and confounders

Two tracks: data systems or training systems

Data systems track

`partitioning`
`join or shuffle strategy`
`caching`
`data loading`

Fix dataset contents and model; verify equivalent results.

Training systems track

`kernel`
`compilation setting`
`communication schedule`
`parallel layout`

Fix model and data; control effective batch and precision.

Key idea

Predict each result with a cost model first. The provided protocol fixes validation data, baseline, budget and tolerances.

How a sound comparison is built

1

Question

One change, named metric.

2

Predict

Direction and cost, before running.

3

Control

List what stays fixed.

4

Repeat

One warmup + three measured runs.

5

Explain

Uncertainty, confounders, limits.

Key idea

Lower token-level loss across different tokenizers isn't comparable; use bits per byte on the same frozen held-out text.

The supplied sweep (retained prototype)

```
MERGES = [0, 16, 32]    # 0 = bytes
DEFAULT_STEPS = 40
GLOBAL_BATCH = 64

$ python 07_capstone/starter/experiment.py \
    --input runs/inputs/bpe \
    --output runs/capstone-r1
```

1. **Checks for you**

   Same held-out checksum and steps × batch budget for every variant.

2. **Smoke first**

   --steps 12 checks the wiring: 768 targets per variant.

3. **Real repeats**

   Separate outputs r1, r2, r3 with the same 40-step budget.

4. **Report limits**

   More merges change vocabulary and parameter count.

What you write, what you get

You write

`Settings and hypotheses`

`starter/experiment.py` · TODO comments mark choices

`Plan and report`

`templates/` · `EXPERIMENT_PLAN.md`, REPORT.md

Provided

- `data_ablation.py` sweep runner
- Run-comparison tool
- Budget and held-out checks
- T1: yours, or the staff copy in provided/

Ground rules

Keep the signatures · don't edit `starter/pipeline_lab` · run through `course.py` or `lab.py`.

Build it in this order

1 — **Pick + plan**
Track, two questions, one-page plan.

2 — **Smoke test**
A 12-step run to check the wiring.

3 — **Measure**
One warmup + three repeats per setting.

4 — **Analyze**
Uncertainty, confounders, negative results.

5 — **Demo**
Raw input → shard → checkpoint → comparison.

```
check capstone
```

Check

Done when: Two comparisons with predictions, fixed controls and three repeats each.

If something goes wrong

One smoke run copied three times
Check: Make independent runs with separate outputs.

Different budgets compared
Check: Same steps $\times$ batch for every variant.

Test data used to choose settings
Check: Choose with validation; test once at the end.

Comparing token-level loss
Check: Use bits per byte on the frozen held-out text.

Claiming a two-host result
Check: Two GPUs on one host is not two hosts.

Ignoring the slow stage
Check: Faster kernels may not matter if ingestion dominates.

The performance study

What makes a comparison believable?

Predict first

Write the prediction and the controls before running anything.

Data systems track
e.g., partition count, loader workers

Training systems track
e.g., kernel or compile on/off, layout

How to measure: Keep hardware allocation consistent, record contention, randomize order when you can, and report total and scoped time.

How you are graded

25	25	35	15
Design	Reproducibility	Analysis	Communication

Capstone = 13% of the course grade · pairs · 6-page report + contribution statement

Check

Done when: Two comparisons with predictions, fixed controls and three repeats each.

Your first ten minutes

```
$ bash setup.sh
$ source .venv/bin/activate
$ python course.py preflight
$ python course.py inputs --output runs/inputs
$ python course.py check capstone
$ python 07_capstone/starter/experiment.py \
    --input runs/inputs/bpe \
    --output runs/capstone-smoke --steps 12
```

Your folder

`07_capstone/`

Handout:
`07_capstone/README.md`